BEOSIN
Blockchain Security

AGV-NFT

Smart Contract Security Audit

No. 202508271019

Aug 27th, 2025

SECURING BLOCKCHAIN ECOSYSTEM

WWW.BEOSIN.COM

Contents

1 Overview **5**

1.1 Project Overview 5

1.2 Audit Overview 5

1.3 Audit Method 5

2 Findings **7**

[AGV-NFT-01] Risk of Centralized Management Authority 8

[AGV-NFT-02] Lack of Checks for WL Start and End Times 9

[AGV-NFT-03] Royalty Fee Lacks Valid Range Check 10

3 Appendix **11**

3.1 Vulnerability Assessment Metrics and Status in Smart Contracts 11

3.2 Audit Categories 14

3.3 Disclaimer 16

3.4 About Beosin 17

Summary of Audit Results

After auditing, 1 Low risk and 2 Info items were identified in the AGV-NFT project. Specific audit details will be presented in the Findings section. Users should pay attention to the following aspects when interacting with this project:

Low	Fixed:1
Info	Fixed:2

Project Description:

The AGV-NFT project consists of 4 primary NFT contracts, which share similar functionality and logic. All contracts support ERC721A efficient NFT minting with upgradeable smart contracts and implement strict role-based access control. Based on AccessControlUpgradeable, the contracts define multiple roles: DEFAULT_ADMIN_ROLE for global management, ADMIN_ROLE for configuring sales parameters and updating whitelists, AGENT_MINTER_ROLE for batch minting, and TREASURER_ROLE for managing fund withdrawals and royalty distribution. This design ensures clear role delegation and well-defined permissions.

In terms of functionality, the contracts feature whitelist verification, USDT payments, dynamic pricing (whitelist, public, and agent prices), batch minting, and royalty configuration (ERC2981). Additionally, they support metadata freezing, sales status toggling, and fund withdrawals (compatible with ETH and ERC20 tokens), offering a comprehensive and flexible solution for NFT sales.

There are, however, several differences among the 4 NFT contracts:

1. Each contract has unique configurations for maximum NFT supply, individual holding limits, allocation proportions for sale types, sale prices, and royalty parameters.
2. In the SeedPass contract, NFTs minted via the agentMint function are exempt from fees.

1 Overview

1.1 Project Overview

Project Name	AGV-NFT
Project Language	Solidity
Platform	Ethereum
Code Base	https://github.com/AGV-Protocol/agvprotocol-contracts
Commit ID	3807234560414fe9974b0f137211b9050894cc5d 6afd5f740402db09c8fb206b0a493e9aa2cd5257

1.2 Audit Overview

Audit work duration: Aug 25, 2025 - Aug 27, 2025

Audit team: Beosin Security Team

1.3 Audit Method

The audit methods are as follows:

1. Formal Verification

Formal verification is a technique that uses property-based approaches for testing and verification. Property specifications define a set of rules using Beosin's library of security expert rules. These rules call into the contracts under analysis and make various assertions about their behavior. The rules of the specification play a crucial role in the analysis. If the rule is violated, a concrete test case is provided to demonstrate the violation.

2. Manual Review

Using manual auditing methods, the code is read line by line to identify potential security issues. This ensures that the contract's execution logic aligns with the client's specifications and intentions, thereby safeguarding the accuracy of the contract's business logic.

The manual audit is divided into three groups to cover the entire auditing process:

The Basic Testing Group is primarily responsible for interpreting the project's code and conducting comprehensive functional testing.

The Simulated Attack Group is responsible for analyzing the audited project based on the collected historical audit vulnerability database and security incident attack models. They identify potential attack vectors and collaborate with the Basic Testing Group to conduct simulated attack tests.

The Expert Analysis Group is responsible for analyzing the overall project design, interactions with third parties, and security risks in the on-chain operational environment. They also conduct a review of the entire audit findings.

3. Static Analysis

Static analysis is a method of examining code during compilation or static analysis to detect issues. Beosin-VaaS can detect more than 100 common smart contract vulnerabilities through static analysis, such as reentrancy and block parameter dependency. It allows early and efficient discovery of problems to improve code quality and security.

2 Findings

Index	Risk description	Severity level	Status
AGV-NFT-01	Risk of Centralized Management Authority	Low	Fixed
AGV-NFT-02	Lack of Checks for WL Start and End Times	Info	Fixed
AGV-NFT-03	Royalty Fee Lacks Valid Range Check	Info	Fixed

Finding Details:

[AGV-NFT-01] Risk of Centralized Management Authority

Severity Level	Low
Type	General Vulnerability
Lines	ComputePass.sol #L305 SeedPass.sol #L285 SolarPass.sol #L305 TreePass.sol #L305

Description	The project's contracts adopt the UUPS proxy pattern, where the `_authorizeUpgrade` function is solely controlled by ADMIN_ROLE. If the private key associated with ADMIN_ROLE is compromised or abused, an attacker could directly replace the contract implementation and gain full control, posing a severe systemic security risk.
	`function _authorizeUpgrade(address newImplementation) internal view override onlyRole(ADMIN_ROLE) {`
	`    require(newImplementation != address(0), "ZeroAddress");`
	`}`

Recommendation	It is recommended to delegate upgrade authorization to a multisignature wallet or DAO to reduce single-point risk.
Status	**Fixed.** The project team has stated that after the contract is deployed, the upgrade authority will be transferred to a multi-signature wallet.

[AGV-NFT-02] Lack of Checks for WL Start and End Times

Severity Level	Info
Type	Business Security
Lines	ComputePass.sol #L195-200 SeedPass.sol #L175-180 SolarPass.sol #L195-200 TreePass.sol #L195-200
Description	The parameters `wlStartTime` and `wlEndTime` define the time window for whitelist users to purchase NFTs. However, there is no strict validation to ensure that `wlStartTime` precedes `wlEndTime`. If `wlStartTime` is set later than `wlEndTime`, whitelist users will be unable to purchase NFTs during the intended period.
Recommendation	It is recommended to add parameter validation in both the constructor and the `setSaleConfig` function to ensure that `wlStartTime` is strictly less than `wlEndTime`, thereby guaranteeing the correctness of the whitelist sale period configuration.
Status	**Fixed.**

[AGV-NFT-03] Royalty Fee Lacks Valid Range Check

Severity Level	Info
Type	General Vulnerability
Lines	ComputePass.sol #L225-228 SeedPass.sol #L205-208 SolarPass.sol #L225-228 TreePass.sol #L225-228
Description	The admin is able to modify the NFT royalty fee via the `setRoyaltyInfo` function. Although the `_setDefaultRoyalty` function already performs basic validation, it is recommended to implement stricter checks within `setRoyaltyInfo`, such as enforcing that the fee cannot exceed 1000 (capping the royalty fee at 10%). This helps prevent misconfigurations that could negatively impact user experience or market liquidity.
Recommendation	It is recommended to implement stricter checks within `setRoyaltyInfo`, such as enforcing that the fee cannot exceed 1000 (capping the royalty fee at 10%).
Status	**Fixed.** The current maximum royalty rate cannot exceed 10%.

3 Appendix

3.1 Vulnerability Assessment Metrics and Status in Smart Contracts

3.1.1 Metrics

In order to objectively assess the severity level of vulnerabilities in blockchain systems, this report provides detailed assessment metrics for security vulnerabilities in smart contracts with reference to CVSS 3.1(Common Vulnerability Scoring System Ver 3.1).

According to the severity level of vulnerability, the vulnerabilities are classified into four levels: "critical", "high", "medium" and "low". It mainly relies on the degree of impact and likelihood of exploitation of the vulnerability, supplemented by other comprehensive factors to determine of the severity level.

Impact / Likelihood	Severe	High	Medium	Low
Probable	Critical	High	Medium	Low
Possible	High	Medium	Medium	Low
Unlikely	Medium	Medium	Low	Info
Rare	Low	Low	Info	Info

3.1.2 Degree of impact

- **Severe**

Severe impact generally refers to the vulnerability can have a serious impact on the confidentiality, integrity, availability of smart contracts or their economic model, which can cause substantial economic losses to the contract business system, large-scale data disruption, loss of authority management, failure of key functions, loss of credibility, or indirectly affect the operation of other smart contracts associated with it and cause substantial losses, as well as other severe and mostly irreversible harm.

- **High**

High impact generally refers to the vulnerability can have a relatively serious impact on the confidentiality, integrity, availability of the smart contract or its economic model, which can cause a greater economic loss, local functional unavailability, loss of credibility and other impact to the contract business system.

- **Medium**

Medium impact generally refers to the vulnerability can have a relatively minor impact on the confidentiality, integrity, availability of the smart contract or its economic model, which can cause a small amount of economic loss to the contract business system, individual business unavailability and other impact.

- **Low**

Low impact generally refers to the vulnerability can have a minor impact on the smart contract, which can pose certain security threat to the contract business system and needs to be improved.

3.1.3 Likelihood of Exploitation

- **Probable**

Probable likelihood generally means that the cost required to exploit the vulnerability is low, with no special exploitation threshold, and the vulnerability can be triggered consistently.

- **Possible**

Possible likelihood generally means that exploiting such vulnerability requires a certain cost, or there are certain conditions for exploitation, and the vulnerability is not easily and consistently triggered.

- **Unlikely**

Unlikely likelihood generally means that the vulnerability requires a high cost, or the exploitation conditions are very demanding and the vulnerability is highly difficult to trigger.

- **Rare**

Rare likelihood generally means that the vulnerability requires an extremely high cost or the conditions for exploitation are extremely difficult to achieve.

3.1.4 Fix Results Status

Status	Description
Fixed	The project party fully fixes a vulnerability.
Partially Fixed	The project party did not fully fix the issue, but only mitigated the issue.
Acknowledged	The project party confirms and chooses to ignore the issue.

3.2 Audit Categories

No.	Categories	Subitems
1	Coding Conventions	Compiler Version Security
		Deprecated Items
		Redundant Code
		require/assert Usage
		Gas Consumption
2	General Vulnerability	Integer Overflow/Underflow
		Reentrancy
		Pseudo-random Number Generator (PRNG)
		Transaction-Ordering Dependence
		DoS (Denial of Service)
		Function Call Permissions
		call/delegatecall Security
		Returned Value Security
		tx.origin Usage
		Replay Attack
		Overriding Variables
		Third-party Protocol Interface Consistency
3	Business Security	Business Logics
		Business Implementations
		Manipulable Token Price
		Centralized Asset Control
		Asset Tradability
		Arbitrage Attack

Beosin classified the security issues of smart contracts into three categories: Coding Conventions, General Vulnerability, Business Security. Their specific definitions are as follows:

- **Coding Conventions**

Audit whether smart contracts follow recommended language security coding practices. For example, smart contracts developed in Solidity language should fix the compiler version and do not use deprecated keywords.

- **General Vulnerability**

General Vulnerability include some common vulnerabilities that may appear in smart contract projects. These vulnerabilities are mainly related to the characteristics of the smart contract itself, such as integer overflow/underflow and denial of service attacks.

- **Business Security**

Business security is mainly related to some issues related to the business realized by each project, and has a relatively strong pertinence. For example, whether the lock-up plan in the code match the white paper, or the flash loan attack caused by the incorrect setting of the price acquisition oracle.

*Note that the project may suffer stake losses due to the integrated third-party protocol. This is not something Beosin can control. Business security requires the participation of the project party. The project party and users need to stay vigilant at all times.

3.3 Disclaimer

The Audit Report issued by Beosin is related to the services agreed in the relevant service agreement. The Project Party or the Served Party (hereinafter referred to as the "Served Party") can only be used within the conditions and scope agreed in the service agreement. Other third parties shall not transmit, disclose, quote, rely on or tamper with the Audit Report issued for any purpose.

The Audit Report issued by Beosin is made solely for the code, and any description, expression or wording contained therein shall not be interpreted as affirmation or confirmation of the project, nor shall any warranty or guarantee be given as to the absolute flawlessness of the code analyzed, the code team, the business model or legal compliance.

The Audit Report issued by Beosin is only based on the code provided by the Served Party and the technology currently available to Beosin. However, due to the technical limitations of any organization, and in the event that the code provided by the Served Party is missing information, tampered with, deleted, hidden or subsequently altered, the audit report may still fail to fully enumerate all the risks.

The Audit Report issued by Beosin in no way provides investment advice on any project, nor should it be utilized as investment suggestions of any type. This report represents an extensive evaluation process designed to help our customers improve code quality while mitigating the high risks in blockchain.

3.4 About Beosin

Beosin is the first institution in the world specializing in the construction of blockchain security ecosystem. The core team members are all professors, postdocs, PhDs, and Internet elites from world-renowned academic institutions. Beosin has more than 20 years of research in formal verification technology, trusted computing, mobile security and kernel security, with overseas experience in studying and collaborating in project research at well-known universities. Through the security audit and defense deployment of more than 2,000 smart contracts, over 50 public blockchains and wallets, and nearly 100 exchanges worldwide, Beosin has accumulated rich experience in security attack and defense of the blockchain field, and has developed several security products specifically for blockchain.

BEOSIN
Blockchain Security

Official Website
https://www.beosin.com

Telegram
https://t.me/beosin

Twitter
https://twitter.com/Beosin_com

Email
service@beosin.com